

Data Structures & Algorithms (Spring 2025) - Problem Set 4 Solution - Ujwal Neethipudi

Question 1: Flask App with UK Parliamentary Constituency Data Visualizations

1(a) Scenario Analysis

For this task, I've created visualizations tailored for government policy advisors who are analyzing voter behavior across the UK to inform future policy decisions. These advisors need to understand:

- 1. How party support varies across different countries within the UK (England, Scotland, Wales)
- 2. How demographic factors like population density relate to voter turnout and party preference

This analysis will help policymakers better understand regional political preferences and factors affecting voter engagement. The insights can be used to design targeted policies addressing region-specific concerns and to develop strategies for increasing democratic participation in areas with historically low engagement.

The policy advisors will use this data to brief ministers on regional political landscapes, identify areas where specific parties have strong or weak support, and better understand the urban-rural divide in British politics. For effective governance, it's crucial for policymakers to understand these patterns to create policies that address regional needs and concerns.

1(b) First Chart: Party Support by Country

My first visualization is a chart showing the support for five major political parties (Conservative, Labour, Liberal Democrats, Green, and Brexit Party) across England, Scotland, and Wales. This chart reveals stark differences in party support between countries - particularly Scotland's distinct voting patterns compared to England and Wales.

This chart is valuable because it helps policymakers quickly identify which parties have significant regional strength, which is essential for understanding how policy proposals might be received in different parts of the UK. The visualization allows for easy comparison between parties and regions, making complex voting data accessible at a glance.

To ensure clarity and accuracy, I've also included a supporting table that presents the exact vote percentages for each party and country.

1(c) Second Chart: Population Density and Voter Turnout Analysis

My second visualization consists of summary tables showing the relationship between population density and voter turnout across UK constituencies. The tables provide comprehensive statistics including average, minimum, and maximum values for both population density and voter turnout in each country, as well as highlighting constituencies with the highest and lowest turnout rates.

This visualization is useful because it helps identify:

- 1. Whether densely populated areas have higher or lower turnout (by comparing constituencies)

2. Which countries have the highest and lowest voter participation
3. Specific constituencies that could be targeted for voter engagement initiatives

Policy advisors can use this information to develop targeted strategies for increasing democratic participation in specific types of areas, particularly focusing on the constituencies with the lowest turnout.

Question 2: Graph Algorithms and Network Optimization

2(a) Adjacency Dictionary and Search Algorithms

I implemented a `GraphTraversal` class with DFS and BFS algorithms. For my randomly generated graph, the adjacency dictionary is:

```
Node 0 -> [4, 5, 8]
Node 1 -> [2, 5]
Node 2 -> [1, 8]
Node 3 -> [4]
Node 4 -> [3, 0, 7]
Node 5 -> [0, 6, 1]
Node 6 -> [5, 9, 7]
Node 7 -> [4, 6]
Node 8 -> [0, 2]
Node 9 -> [6]
```

Starting from node 0, my search algorithms produced these traversals:

- DFS Traversal: [0, 4, 3, 7, 6, 5, 1, 2, 8, 9]
- BFS Traversal: [0, 4, 5, 8, 3, 7, 6, 1, 2, 9]

Observations: DFS and BFS reveal fundamentally different approaches to graph exploration. DFS dives deep along a single path, pushing it all the way to its conclusion before backtracking. This creates a traversal pattern that appears to jump around the graph. BFS, in contrast, explores in concentric circles around the starting node, visiting all nearest neighbors before moving outward. This creates a more structured, level-by-level exploration that discovers nodes in order of their distance from the starting point.

2(b) Economic Network Optimization

i. Optimization Problem Formulation

The social planner's problem can be formulated as:

Maximize: $\sum \log(1/d_i + x_i)$

Subject to:

- $\sum x_i = B_t$
- $x_i \geq 0$ for all i

Where:

- d_i is the degree of node i
- x_i is the budget allocated to node i
- $B_t = 10$ is the total available budget

ii. Optimal Budget Allocation Solution

The optimal budget allocation can be derived using the Lagrangian method. For each node, the optimal allocation is:

$$x_i^* = \max(0, 1/\lambda - 1/d_i)$$

Where λ is the Lagrange multiplier that satisfies:

$$\sum x_i^* = B_t$$

This leads to:

$$\lambda = n / (B_t + \sum (1/d_i))$$

Where n is the number of nodes in the network.

iii. Budget Allocation Chart for $B_t = 10$

The calculated allocations for each node in my generated graph are:

- Node 0 (degree 3): 1.2000
- Node 1 (degree 2): 1.0333
- Node 2 (degree 2): 1.0333
- Node 3 (degree 1): 0.5333
- Node 4 (degree 3): 1.2000
- Node 5 (degree 3): 1.2000
- Node 6 (degree 3): 1.2000
- Node 7 (degree 2): 1.0333
- Node 8 (degree 2): 1.0333
- Node 9 (degree 1): 0.5333

The key insight from these allocations is that nodes with lower degrees receive smaller budget allocations compared to nodes with higher degrees. This demonstrates the principle of diminishing returns - resources are optimally allocated to nodes with the greatest marginal benefit.

iv. Modified Optimization Problem with Minimum Payoff

To maximize aggregate welfare while ensuring a minimum payoff $\phi_{\min}$ for each node, the optimization problem becomes:

$$\text{Maximize: } \sum \log(1/d_i + x_i)$$

Subject to:

- $\sum x_i = B_t$
- $x_i \geq 0$ for all i

- $\log(1/d_i + x_i) \geq \phi_{\min}$ for all i

The third constraint can be rewritten as:

- $x_i \geq \max(0, e^{\phi_{\min}} - 1/d_i)$ for all i

This modified problem ensures that every node receives at least enough budget to achieve the minimum required payoff $\phi_{\min}$, while still optimizing the overall welfare of the network subject to the total budget constraint.

GitHub Pull Request URL: https://github.com/yourusername/my_flask_app_assignment_4/pull/1